1. What is Full Stack Development?

Development that covers both the frontend (what the user sees) and backend (server/business logic), including database handling — i.e., a full stack developer can build an entire application end-to-end.

2. Frontend vs Backend vs Database.

Frontend is the user-facing part (UI/UX) built with HTML, CSS, JS, React. Backend handles logic, APIs, and requests (Java, Spring Boot). Database stores and manages persistent data (SQL/MySQL).

3. Client-server architecture.

A model where the client (browser/app) sends requests to a server, which processes them (often interacting with a database) and sends back a response.

4. How does a web application work, step by step.

User action in browser → request sent to server → server processes request (business logic, DB query if needed) → server sends response → browser renders the result.

5. Introduction to Java Full Stack Development?

Java Full Stack Development means building a complete web application — from the user interface to the server logic to the database — using Java as the core backend technology, paired with modern frontend tools.

A "full stack" developer works across three layers:

1. Frontend (Client-side)

What the user sees and interacts with in the browser.

- **HTML** – structure/content of web pages
- **CSS** – styling and layout
- **JavaScript** – interactivity and dynamic behavior
- **React.js** – a JavaScript library for building fast, component-based user interfaces

2. Backend (Server-side)

Handles business logic, processes requests, and connects frontend to the database.

- **Java** – the core programming language for backend logic
- **Spring Boot** – a Java framework that simplifies building REST APIs and backend services (handles routing, security, dependency management, etc.)

3. Database

Stores and manages the application's data.

- **SQL** (e.g., MySQL) – used to create, read, update, and delete data

6. Overview of HTML, CSS, JavaScript, React.js, Java, Spring Boot, and SQL?

HTML (HyperText Markup Language)

The skeleton/structure of a web page

Defines elements like headings, paragraphs, images, forms, buttons

Not a programming language — a markup language

CSS (Cascading Style Sheets)

Controls the visual presentation: colors, fonts, spacing, layout

Makes HTML pages look good and responsive across devices

Can be written inline, internally, or in external .css files

JavaScript (JS)

Adds interactivity and dynamic behavior to web pages

Handles things like form validation, button clicks, animations, API calls

Runs in the browser (client-side)

React.js

A JavaScript library (by Meta/Facebook) for building user interfaces

Component-based: UI is broken into reusable pieces (e.g., Navbar, Button, Card)

Uses a Virtual DOM for fast rendering/updates

Popular for building Single Page Applications (SPAs)

Backend Technologies

Java

A robust, object-oriented, platform-independent programming language

Widely used for enterprise-level backend applications

Known for stability, scalability, and strong community support

Spring Boot

A Java framework that simplifies backend development

Removes boilerplate configuration (auto-configuration, embedded server)

Used to build REST APIs that the frontend communicates with

Handles things like routing, security, database connections, dependency injection

---

Database Technology

SQL (Structured Query Language)

Used to interact with relational databases (e.g., MySQL, PostgreSQL)

Performs CRUD operations: Create, Read, Update, Delete

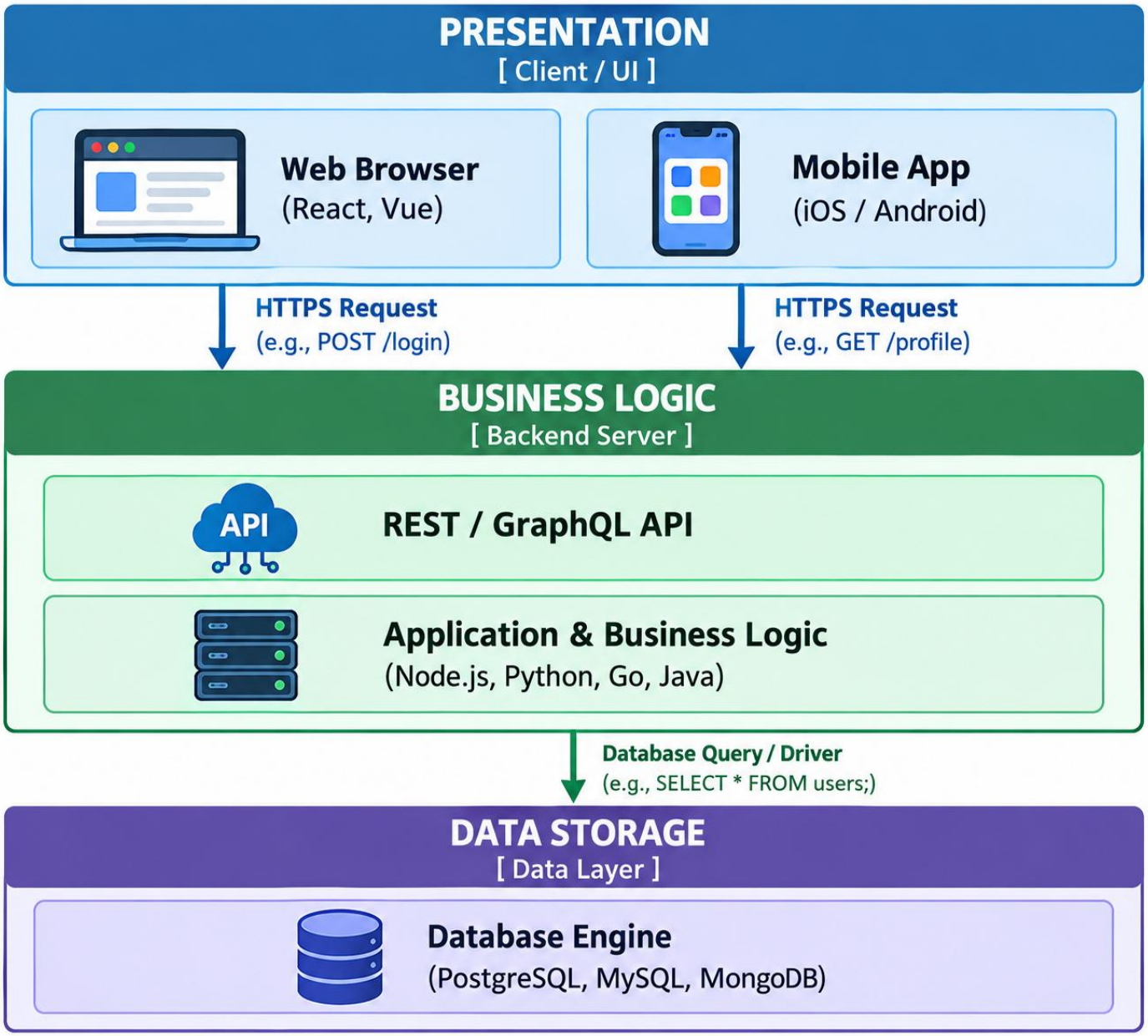
Organizes data into tables with rows and columns

Example: SELECT * FROM users WHERE id = 1;

How They Work Together

• Layer	• Technology	• Role
• Frontend	• HTML, CSS, JS, React	• What the user sees & interacts with
• Backend	• Java, Spring Boot	• Processes logic, serves data via APIs
• Database	• SQL (MySQL)	• Stores and retrieves data

7. Create a simple architecture diagram showing frontend, backend, and database communication.


PRESENTATION
[Client / UI]
Web Browser
(React, Vue)
Mobile App
(iOS / Android)
HTTPS Request
(e.g., POST /login)
HTTPS Request
(e.g., GET /profile)
BUSINESS LOGIC
[Backend Server]
API
REST / GraphQL API
Application & Business Logic
(Node.js, Python, Go, Java)
Database Query / Driver
(e.g., SELECT * FROM users;)
DATA STORAGE
[Data Layer]
Database Engine
(PostgreSQL, MySQL, MongoDB)